Conceptos básicos para la programación de aplicaciones Android.

Caso práctico



Mikhail (Pexels)

Una vez que el equipo de trabajo tiene el IDE instalado y algunas ideas de cómo se pueden programar las apps y probarlas en dispositivos virtuales y reales, es necesario adquirir los conceptos básicos en el desarrollo de las aplicaciones para Android.

- Buenos días -saluda Alda.

- Buenos días -responden María, Juan y Pedro.

- Hoy comenzaremos viendo las distintas fases de la programación por las que es necesario pasar para convertir nuestro código java en una aplicación ejecutable en un terminal Android -indica Ada-. Después estudiaremos las peculiaridades de la **interfaz de usuario** de las aplicaciones Android, los **controles principales** y el **manejo de sus eventos** más importantes, así como algunos conceptos interesantes para su diseño como el manejo del **foco**.

- Perfecto -responde Juan-. Además seguro que los conceptos previos que tenemos sobre Programación Orientada a Objetos nos van a ser muy útiles.

- Por supuesto -afirma Ada-. Es muy importante conocer los distintos **controles de entrada** de datos que nos van a permitir interactuar con nuestros usuarios. Os voy a asignar algunos pequeños proyectos para que entendáis su funcionamiento.

-¡Genial! -exclama María.

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN Y FORMACIÓN PROFESIONAL

Ministerio de Educación y Formación Profesional. (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación y Formación Profesional.

Aviso Legal

1.- Fases de la programación.

Caso práctico



Mikhail (Pexels)

Todos los usuarios de dispositivos móviles sabemos que cuando deseamos instalar una app en nuestro dispositivo acudimos a la "tienda" de nuestro proveedor. En caso de dispositivos Apple abriremos *Apple Store* y en caso de dispositivos Android abriremos *Google Play Store*.

Estas plataformas son en realidad repositorios de paquetes que albergan las aplicaciones.

- Como ya imagináis, desde que se desarrolla una app hasta que se publica en uno de estos repositorios de publicación, es necesario pasar por una serie de fases -explica Ada-. Estas fases forman el ciclo de vida de una app móvil.

- Entiendo que como nosotros vamos a desarrollar aplicaciones para Android, el repositorio de publicación que le corresponde es Google Play Store. -pregunta Pedro-. ¿Es así?.

- Estás en lo cierto, Pedro -confirma Ada-. En el caso de Android, el fichero de manifiesto (**Androidmanifiest.xml**) juega un papel muy importante en la definición de los metadatos asociados a la app y al propio paquete.

- ¿Cuáles son esos metadatos? -pregunta María.

- Son, por ejemplo, las actividades que forman la aplicación, los permisos que se necesitan y otra gran cantidad de datos relativos a la propia aplicación -responde Ada.

- ¡Pues vamos a ver las fases de la programación! -exclama sonriente Juan.

En la unidad anterior ya se ha avanzado algo sobre las herramientas que podrían utilizarse para desarrollar una aplicación para dispositivo móvil. Como ya has debido ver anteriormente en el módulo de **Programación**, normalmente una aplicación Java podrá ser desarrollada utilizando herramientas a través de la línea de comandos o bien mediante la utilización de un entorno de desarrollo integrado (**IDE**). Sea cual sea el método que se utilice para el desarrollo del código, podrás distinguir las siguientes fases de construcción para una aplicación basada en Java con **Android Studio**:

- Desarrollo del código
- Compilación
- Preverificación
- Empaquetamiento
- Ejecución
- Depuración

En nuestro caso, el entorno de desarrollo que usaremos será Android Studio. Por otro lado, en la unidad anterior ya vimos cómo instalar esta herramienta para poder desarrollar aplicaciones Android y cómo compilar una aplicación básica para poder ejecutarla en el emulador y en un dispositivo real.

1.1.- Desarrollo del código: codificación.

Esta es la fase en la que vas a escribir las líneas de **código** de tu aplicación. Si no estuvieras utilizando un IDE (en este caso Android Studio) necesitarías un editor de texto para escribir y almacenar los archivos java que contendrán las clases, así como los ficheros XML que se van a ir generando. En este caso, la cosa es tan sencilla como utilizar el editor de textos proporcionado por Android Studio.

Si el entorno integrado de desarrollo dispone de asistentes para la creación de aplicaciones, parte de ese código podrá ser generado automáticamente por el asistente, especialmente todo lo que concierne a la interfaz gráfica de usuario y especificación de recursos que utilizará el sistema. Android Studio dispone de estos asistentes y los usaremos con frecuencia para facilitarnos el trabajo.

1.2.- Compilación y preverificación.

El proceso de **compilación**, es decir, de generación de un archivo binario de tipo bytecode (class) a partir de otro en código **Java** ya lo has estudiado en otros módulos y no cambia en absoluto. Una vez que tengas el código fuente de tu programa escrito en uno o varios archivos con extensión **.java**, la compilación de aplicaciones se puede realizar con el programa *javac* incluido en el software de desarrollo de Java o bien mediante la utilización del entorno integrado de Android Studio.

Los procesos de **verificación** durante la generación de código se encargan de realizar revisiones de seguridad, integridad y cumplimiento de los estándares. Debido a las importantes restricciones existentes en los dispositivos móviles, la **verificación** de las clases tiene dos etapas:

- **Preverificación**, que se realiza como un paso más de la generación de los ficheros binarios.
- **Verificación**, llevada a cabo por la máquina virtual durante la ejecución.

1.3.- Empaquetamiento.

Esta fase es la que se encarga de empaquetar la aplicación con todas las clases y recursos que vaya a necesitar, dejándola lista para ser descargada en un dispositivo que la instale.

Si tratamos de generar una aplicación de Android tendremos que exportar nuestro proyecto Android Studio a un fichero APK (***A**ndroid **A**pplication Pac**k**age*, significado en español: *Paquete de Aplicación Android*). El nombre APK se refiere a la extensión que tiene el fichero compacto y empaquetado que contiene todo el código de la aplicación.

En este proceso de exportación hay dos alternativas posibles según como se haya realizado el proceso:

a. Generación de Fichero **APK** de tipo **Debug**. Consistirá en un fichero APK destinado a poder hacer de manera local pruebas de funcionamiento de la aplicación, instalándolo en dispositivos virtuales o en nuestro propio dispositivo físico, pero todo ello en un ámbito restringido. En ningún caso este fichero es válido para subirlo a **Play Store Google**, aún así si podré instalarlo de manera remota en mi dispositivo móvil siempre que haya habilitado el dispositivo para permitir instalación de aplicaciones ***no firmadas***.
b. Fichero APK firmado digitalmente (**signed APK**). Consistirá en un fichero que ha sido firmado con un certificado digital, de manera que este fichero garantiza la autoría del mismo por la persona que desarrolló la aplicación. Este tipo de fichero si puede ser subido a **Play Store Google** para publicar una app. De hecho, si no está firmado no puede subirse y cualquier actualización que hagamos de la app en Play Store, deberá ser firmada con el mismo certificado siempre.

1.4.- El fichero de manifiesto.

Cuando generamos cualquier aplicación, siempre llevará incorporada un fichero de "manifiesto" que contiene información de configuración de la aplicación en cuestión. Este archivo describe el contenido de la aplicación, indicando las actividades que contiene, así como entre otras cosas permisos para acceder a ciertos recursos y funciones del dispositivo, y consiste un archivo de texto con una estructura de una línea por atributo, donde los valores de los atributos son especificados de la forma "atributo: valor".

Según Android Developer :

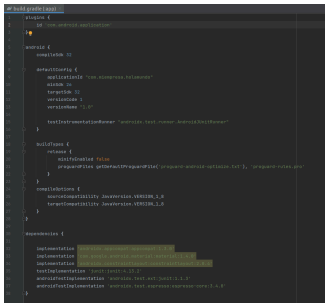
Todas las aplicaciones deben tener un fichero **AndroidManifest.xml** (con ese nombre exacto) en el directorio *nombre_proyecto/app/src/main*. El archivo de manifiesto proporciona información esencial sobre tu aplicación al sistema Android, información que el sistema debe tener para poder ejecutar el código de la app.

Entre otras cosas, el fichero de manifiesto hace lo siguiente:

- Nombra el paquete de Java para la aplicación. El nombre del paquete sirve como un identificador único para la aplicación. Cuando lo subas a Play Store Google debe ser único globalmente.
- Describe los componentes de la aplicación, como las actividades, los servicios, los receptores de mensajes y los proveedores de contenido que la integran. También nombra las clases que implementa cada uno de los componentes y publica sus capacidades, como los mensajes **Intent** con los que pueden funcionar. Estas declaraciones notifican al sistema Android los componentes y las condiciones para el lanzamiento.
- Declara los permisos debe tener la aplicación para acceder a las partes protegidas de una API e interactuar con otras aplicaciones. También declara los permisos que otros deben tener para interactuar con los componentes de la aplicación.
- Funciones Hardware y Software que requiere la aplicación, lo que implica que se pueda instalar o no en un dispositivo.
- Enumera las clases **Instrumentation** que proporcionan un perfil y otra información mientras la aplicación se ejecuta. Estas declaraciones están en el manifiesto solo mientras la aplicación se desarrolla y se quitan antes de la publicación de esta.

Debes conocer

Además del archivo AndroidManifest.xml existen otro archivos importantes cómo el archivo *nombre_proyecto/app/build.gradle* dónde se indican algunos parámetros importantes de la aplicación Android



Google Developers (Uso educativo nc)

- Declara una lista de plugins que deseamos incluir en la aplicación.
- Declara varios parámetros importantes de la aplicación, algunos de ellos:
  - Identificador (es el nombre del paquete java) y versión de la aplicación.
  - Versión del SDK con la que se va a realziar la compilación.
  - Versión mínima de SDK necesaria en el terminal para que la app funcione y la versión objetivo (*target*) para la que se ha diseñado.
- Enumera las bibliotecas de las que la aplicación depende (*dependencies*).

Un plugin es aquella aplicación que, en un programa informático, añade una funcionalidad adicional o una nueva característica al software. En nuestro idioma, por lo tanto, puede nombrarse al plugin como un complemento.

1.5.- Estructura del fichero de manifiesto.

A continuación se muestra la estructura general de dicho fichero (todos los enlaces indicados en el código llevan a enlaces de Android Developer y se abren en ventana nueva).

Estructura general del fichero de manifiesto

```
xml version="1.0" encoding="utf-8"?>

  /> /> /> /> /> /> /> />
/> /> />


    />
    />
    />

  />

  . . .
  />

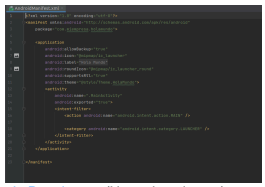
  . . .

  . . .
  />

  />
  />
  />

/>
```

En la estructura anterior se presentan todas las posibles etiquetas que podrían utilizarse. En la práctica usaremos muchas menos. Así, en una aplicación básica, con una solo actividad (ficheros asociados **MainActivity.java** y **activity_main.xml**) el fichero **AndroidManifest.xml** presenta este aspecto, como vemos, bastante más simple:



Google Developers (Uso educativo nc)

La siguiente lista contiene todos los elementos que pueden aparecer en el fichero de manifiesto (en orden alfabético):

-
-
-
-
-

- 
- 
- 
- 
- 
- 
- 
- 
- 
- 
- 
- 
- 
- 
- 
- 
- 
- 

Nota: Estos son todos los elementos permitidos; no puedes agregar elementos o atributos propios.

1.6.- Convenciones del fichero de manifiesto.

A todos los elementos y atributos del fichero de manifiesto se les aplican una serie de convenciones y reglas generales:

Elementos

Solo se requieren los elementos y . Cada uno de ellos debe estar presente y solo puede ocurrir una vez. La mayoría de los otros elementos pueden ocurrir varias veces o ninguna. Sin embargo, al menos algunos de ellos deben estar presentes para que el archivo de manifiesto sea útil.

Si un elemento contiene algo, serán otros elementos. Todos los valores se establecen mediante atributos, en lugar de datos de caracteres en un elemento.

Por lo general, los elementos de un mismo nivel no están ordenados. Por ejemplo, los elementos , y se pueden alternar en cualquier secuencia. Hay dos excepciones importantes a esta regla:

- Un elemento debe seguir el en el que funciona como alias.
- El elemento debe ser el último dentro del elemento . En otras palabras, la etiqueta de cierre de debe aparecer inmediatamente antes de la etiqueta de cierre de .

Atributos

En términos formales, todos los atributos son opcionales. Sin embargo, existen algunos atributos que deben especificarse para que un elemento cumpla con su propósito. Usa la documentación como guía. Para los atributos realmente opcionales, se menciona un valor predeterminado o se indica lo que sucede si no se especifica ningún valor.

Excepto para algunos atributos del elemento raíz , todos los nombres de los atributos comienzan con un prefijo **android:**. Por ejemplo, **android:alwaysRetainTaskState**. Debido a que el prefijo es universal, en la documentación suele omitirse cuando se hace referencia a los atributos por nombre.

Declaración de nombres de clases

Muchos elementos corresponden a objetos Java, incluidos los elementos para la aplicación en sí (el elemento) y sus componentes principales: actividades ( ), servicios ( ), receptores de mensajes ( ) y proveedores de contenido ( ).

Si defines una subclase, como lo harías generalmente para las clases de componentes (Activity , Service , BroadcastReceiver y ContentProvider), la subclase se declara mediante un atributo **name**. El nombre debe incluir la designación completa del paquete.

Ejemplo

Una subclase Service podría declararse de la siguiente manera:

```
...

...
```

Sin embargo, si el primer carácter de la cadena (*string*) es un punto, el nombre del paquete de la aplicación (tal como lo especifica el atributo **package** del elemento) se adjunta a la cadena (*string*).

Ejemplo

La siguiente asignación es la misma que se mostró en el ejemplo anterior.

```
...
...
```

Cuando inicia un componente, el sistema Android crea una instancia de la subclase mencionada. Si no se especifica una subclase, crea una instancia de la clase base.

Varios valores

Si se puede especificar más de un valor, el elemento se repite casi siempre, en lugar de enumerar varios valores en un único elemento.

Ejemplo

En un **filtro de objetos Intent** se pueden enumerar varias acciones:

Valores de recursos

Algunos atributos tienen valores que se pueden mostrar a los usuarios, como una etiqueta y un ícono para una actividad. Los valores de estos atributos deben localizarse y establecerse desde un recurso o tema. Los valores de los recursos se expresan en el siguiente formato:

@[package:]type/name

Puedes omitir el nombre de paquete (*package)* si el recurso se encuentra en el mismo paquete de la aplicación.

type es un tipo de recurso, como una cadena (*string)* o un elemento de diseño (*drawable*).

name es el nombre que identifica al recurso específico.

A continuación, te mostramos un ejemplo:

Los valores de un tema se expresan de manera similar, aunque con un ? al inicio en lugar de @:

?[package:]type/name

Valores de cadenas (strings)

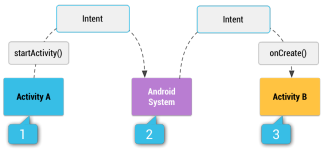
Cuando el valor de un atributo sea una cadena (*string*), debes usar doble barra invertida (\\) para diferenciarlo de los caracteres, como en el caso del uso de \\n para una nueva línea o \\uxxxx para un carácter Unicode.

1.7.- Funciones del fichero de manifiesto.

Filtros de intenciones (*intent filters*)

Los componentes centrales de una aplicación (como sus actividades, servicios y receptores de mensajes) se activan mediante **intenciones** (***intents)***. Las intenciones se representan mediante la clase Intent .

Un objeto de mensajería **intent**:



Google Developers (Uso educativo nc)

- Es una instancia de la clase **Intent** y podemos usarlo para solicitar una acción desde otro componente de la app.
- Facilita la comunicación entre componentes de distintas maneras.
- Describe una acción deseada, los datos en los que se debe actuar, la categoría del componente que debe realizar la acción y otras instrucciones pertinentes.
- Es respondido apropiadamente porque Android localiza un componente para ello, iniciando una instancia nueva del mismo si es necesario y le pasa el objeto de mensajería (ver Intent **).**

Los componentes anuncian los tipos de objetos de mensajería (***intents*)** a los que pueden responder mediante **filtros de intenciones (intents)**. Debido a que el sistema Android debe aprender las intenciones (***intents*)** que puede manejar un componente antes de iniciarlo, los **filtros de intenciones (*intents*)** se especifican en el manifiesto como elementos . Un componente puede tener cualquier cantidad.

Ejemplo

Ejemplo de filtro de intenciones (declaración de actividad con un filtro de intenciones para recibir una intención ACTION_SEND cuando el tipo de datos es texto):

Una **intención** (***intent*)** que nombre explícitamente un componente de destino activará ese componente; el filtro no participa. Una **intención** (***intent*)** que no nombra explícitamente un componente de destino puede activar un componente únicamente si pasa por uno de los filtros del componente.

El uso más importante de las **intenciones** (***intents***) es para el lanzamiento de actividades, donde se puede pensar que son el "pegamento" entre las **actividades** (***activities***).

Por tanto, una **intención** (***intent***) es una estructura de datos pasiva que representa una descripción abstracta de una acción a ser realizada.

Íconos y etiquetas

Varios elementos tienen atributos **icon** y **label** para un icono pequeño y una etiqueta de texto que se pueden mostrar a los usuarios. Algunos también tienen un atributo **description** para un texto explicativo más extenso que también se puede mostrar en pantalla. Por ejemplo, el elemento <permission> tiene estos tres atributos. Cuando se pregunta al usuario si otorga permiso a una aplicación que lo solicitó, se le proporciona un icono que representa el permiso, el nombre del permiso y una descripción de lo que implica.

En cada caso, el icono y la etiqueta establecidos en un elemento contenedor se convierten en la configuración predeterminada de **icon** y **label** para todos los subelementos del contenedor. Por lo tanto, el icono y la etiqueta establecidos en el elemento <application> son el icono y la etiqueta predeterminados para cada componente de la aplicación. Igualmente, el icono y la etiqueta establecidos para un componente, como un elemento <activity> , son la configuración predeterminada de cada elemento <intent-filter> del componente. Si un elemento

<application> establece una etiqueta, pero una actividad y su filtro de intenciones (*intents*) no lo hacen, la etiqueta de la aplicación se considera como la etiqueta de la actividad y del filtro de intenciones (*intents*).

El icono y la etiqueta establecidos para un filtro de intenciones (*intents*) se usan para representar un componente cuando el componente se presenta al usuario y cumple la función indicada en el filtro. Por ejemplo, un filtro con las configuraciones **android.intent.action.MAIN** y **android.intent.category.LAUNCHER** indica que la actividad inicia una aplicación. Es decir, como una que se debe mostrar en el lanzador de la aplicación. El icono y la etiqueta establecidos en el filtro se muestran en el lanzador.

Permisos

Un *permiso* es una restricción que limita el acceso a una parte del código o a datos en el dispositivo. La limitación se impone para proteger datos y códigos claves que podrían usarse incorrectamente para distorsionar o afectar la experiencia del usuario.

Cada permiso se identifica con una etiqueta única. Con frecuencia, la etiqueta indica la acción que está restringida. A continuación, se nombran algunos permisos definidos por Android:

- **android.permission.CALL_EMERGENCY_NUMBERS**
- **android.permission.READ_OWNER_DATA**
- **android.permission.SET_WALLPAPER**
- **android.permission.DEVICE_POWER**

Una funcionalidad se puede proteger con un solo permiso.

Si una aplicación necesita acceso a una característica protegida por un permiso, debe declarar que requiere el permiso con un elemento <uses-permission> en el manifiesto. Cuando la aplicación se instala en el dispositivo, el instalador determina si otorgará el permiso solicitado controlando las autoridades que firmaron los certificados de la aplicación y, en algunos casos, preguntándole al usuario. Si se otorga el permiso, la aplicación puede usar las características protegidas. De lo contrario, los intentos de acceder a esas características fallarán y no habrá notificaciones para el usuario.

Una aplicación también puede proteger sus propios componentes con permisos. Puede emplear cualquiera de los permisos definidos por Android (enumerados en android.Manifest.permission) o declarados por otras aplicaciones. También puede definir sus propios permisos. Un permiso nuevo se declara con el elemento .

Ejemplo

Una actividad se puede proteger de la siguiente manera:

```
...

...
```

Ten en cuenta que, en este ejemplo, el permiso **DEBIT_ACCT** no solo se declara con el elemento <permission>; su uso también se solicita con el elemento <uses-permission>. Debes solicitar su uso para que otros componentes de la aplicación inicien la actividad protegida, aunque la protección se imponga a través de la propia aplicación.

En el mismo ejemplo, si el atributo **permission** estuviera establecido en un permiso declarado en otro lugar (como **android.permission.CALL_EMERGENCY_NUMBERS**), no sería necesario volver a declararlo con un elemento <permission>. Sin embargo, seguiría siendo necesario solicitar su uso con <uses-permission>.

El elemento <permission-tree> declara un espacio de nombres para un grupo de permisos definidos en código y <permission-group> define una etiqueta para un conjunto de permisos (aquellos declarados en el manifiesto con elementos y aquellos declarados en otro lugar). Esto solo afecta la agrupación de los permisos cuando se presentan al usuario. El elemento <permission-group> no especifica los permisos pertenecientes al grupo, pero le asigna el nombre. Puedes disponer un permiso en el grupo si asignas el nombre de este al elemento <permission> del atributo permission-group.

Bibliotecas

Todas las aplicaciones están vinculadas con la biblioteca predeterminada de Android, que incluye los paquetes básicos para crear aplicaciones (con clases comunes como **Activity**, **Service, Intent**, **View**, **Button**, **Application** y **ContentProvider**).

Sin embargo, algunos paquetes se encuentran en sus propias bibliotecas. Si tu aplicación usa código de alguno de estos paquetes, debe solicitar explícitamente su vinculación con ellos. El manifiesto debe contener un elemento **<uses-library>** separado para nombrar cada una de las bibliotecas. En la documentación del paquete puedes encontrar el nombre de la biblioteca.

1.8.- Instalación en un dispositivo real.

Para la instalación de una aplicación en un dispositivo real existen varias alternativas, pero has de tener en cuenta que en el fondo se trata de copiar el archivo APK de la aplicación en la memoria del dispositivo y realizar su instalación en él.

Si el archivo APK se encuentra en un servidor, sería suficiente con descargarlo mediante el navegador web del dispositivo o bien utilizar un gestor de aplicaciones y a partir de ese momento sería el propio dispositivo el que se encargaría de descargar e instalar el archivo APK.

Si el archivo APK no está en un servidor, sino que lo tienes en tu ordenador, recién compilado y en espera de pruebas, puedes copiarlo a la memoria del dispositivo de la manera que te resulte más cómoda:

- Mediante la transferencia del archivo APK por Bluetooth que integra el propio dispositivo.
- A través de una conexión USB y copiando el archivo APK en la memoria del dispositivo.
- Copiando el archivo APK en alguna memoria externa soportada por el dispositivo (una tarjeta SD por ejemplo) e insertándole esa memoria.

1.9.- Ejecución.

La ejecución de una aplicación puede realizarse bien en un emulador (basta con ejecutarla en Android Studio para que ésta sea lanzada en el emulador que tengamos configurado en el proyecto) bien en un dispositivo real. Es casi seguro que el aspecto final será distinto en ambos casos, aunque el funcionamiento debería ser prácticamente idéntico (a veces podría haber algunas diferencias debido a la máquina virtual que haya instalada en el dispositivo). También es muy probable que haya cambios de aspecto entre distintos dispositivos (dependiendo de la máquina virtual que tengan instalada).

En el momento en que la aplicación se ponga a funcionar se irán sucediendo cada uno de los estados **en pausa, activo** y **destruido** y se irán ejecutando todos los métodos que hayamos añadido.

2.- Interfaces de usuario.

Caso práctico

Una vez que el equipo es conocedor de las fases que tendrá que completar para convertir el programa desarrollado con Android Studio en una aplicación descargable en un dispositivo, es el momento de pensar en el diseño gráfico que permitirá a los usuarios usar la aplicación.



Mikhail (Pexels)

- El diseño gráfico de una aplicación es muy importante -explica Ada-. El diseño será lo que conseguirá, o no, que nuestra aplicación sea usable en distintos dispositivos y permita la interacción con el usuario.

- Claro -responde Juan-, un mal diseño gráfico puede hacer inservible una aplicación.

- Exacto -confirma Ada-. Para ello necesitáis conocer algunos parámetros del diseño o layouts.

- ¿Qué lenguaje se utiliza para el diseño de las aplicaciones Android? -pregunta Juan.

- Vas a tener suerte -responde Ada-, todo el diseño (layout) de la aplicación Android se configura mediante el lenguaje XML.

- ¡Qué bien! -exclama Juan-, pondré en práctica todo lo aprendido en el módulo de Lenguaje de Marcas.

2.1.- Información general de la Interfaz de usuario.

La interfaz gráfica de usuario es la parte de un programa que permite al usuario interaccionar con él. En el caso de Java SE dispones de dos extensas bibliotecas gráficas (AWT y Swing, que son parte de las JFC), pero para el caso de Android Studio o Java ME, dada la gran cantidad de restricciones que tienen los dispositivos móviles, hubo que diseñar un pequeño conjunto de clases específicas que cumplieran esas restricciones.

La interfaz de usuario (IU) de tu app es todo aquello que el usuario puede ver y todo aquello con lo que este puede interactuar. Android ofrece una variedad de componentes de IU previamente compilados, como objetos de diseño estructurados y controles de IU que te permiten compilar la interfaz gráfica de usuario para tu app. Android también ofrece otros módulos de IU para interfaces especiales, como diálogos, notificaciones y menús.

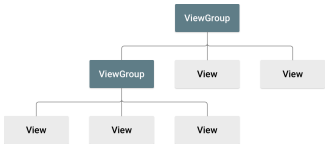
Según Android Developers todos los elementos de la interfaz de usuario de una app para Android están desarrollados con objetos **View** y **ViewGroup** . Un objeto **View** dibuja algo en la pantalla con lo que el usuario puede interactuar. Un objeto **ViewGroup** agrupa otros objetos **View** (y **ViewGroup**) para definir el diseño de la interfaz.

Android proporciona una colección de subclases **View** (y **ViewGroup**) que te ofrecen controles de entrada comunes (como los botones y los campos de texto) y varios modelos de diseño (como un diseño lineal o relativo).

2.2.- Diseño de la interfaz de usuario.

El diseño (*layout*) define la estructura de la interfaz de usuario de tu app. Todos los elementos en el *layout* se construyen usando una jerarquía de objetos **View** y **ViewGroup** , como se muestra en la ilustración más abajo.

Una vista (objeto View) normalmente dibuja algo que el usuario puede ver y con lo que puede interactuar mientras que un grupo de vista (objeto ViewGroup) es un contenedor invisible que define la estructura de diseño de otros objetos View y ViewGroup. Este árbol de jerarquía puede ser tan simple o complejo como lo necesites (pero la simplicidad es lo mejor para el rendimiento).



Google Developers (Uso educativo nc)

Los objetos View se denominan controles (*widgets*) y podemos crear instancias de muchas subclases como por ejemplo **Button** o **TextView** . Los objetos ViewGroup se denominan estructuras de diseño (*layouts*) y pueden ser de distintos tipos los cuales proporcionan diferentes estructuras de diseño como **LinearLayout** o **ConstraintLayout** .

Para definir el diseño, puedes crear instancias de objetos ViewGroup y View en el código y desarrollar un árbol, pero la manera más sencilla y efectiva de definir el diseño consiste en utilizar un archivo XML. XML ofrece una estructura en lenguaje natural para el diseño, similar a HTML.

El nombre de un elemento XML para una vista se relaciona con la clase de Android que representa. Por lo tanto, un elemento <TextView> crea un control gráfico (***widget***) TextView en la IU, y un elemento <LinearLayout> crea un grupo de vista LinearLayout.

Ejemplo

Un diseño vertical simple con una vista de texto y un botón tiene esta apariencia:



Cuando cargas un recurso de diseño en la app, Android inicia cada nodo del diseño en un objeto de tiempo de ejecución que puedes usar para definir comportamientos adicionales, consultar el estado del objeto o modificar el diseño.

Para consultar una guía completa sobre la creación de un diseño de IU, consulta el apartado siguiente sobre Diseños (layouts).

2.3.- Otros componentes de la interfaz de usuario.

Además de los objetos **View** y **ViewGroup**. Android proporciona varios componentes de app que ofrecen un diseño de IU estándar, en los cuales solo tienes que definir el contenido. Cada uno de estos componentes de IU tienen un conjunto único de clases en la API y se describen en los apartados de la documentación correspondientes, como Adición de la barra de app , Cuadros de diálogo y Notificaciones de estado .

2.4.- Diseños (layouts).

Un diseño (*layout*) define la estructura visual para una interfaz de usuario, como la IU para una **actividad** o control (*widget*) de una app. Puedes definir un diseño (*layout*) de dos maneras:

- **Definir elementos de la IU en XML**. Android proporciona un vocabulario XML simple que coincide con las clases y subclases de vistas, como las que se usan para controles (*widgets*) y diseños.
- **Instanciar elementos de diseño en tiempo de ejecución**. Tu aplicación puede crear objetos **View** y **ViewGroup** (y manipular sus propiedades) programáticamente.

El framework de Android te ofrece la flexibilidad de usar uno de estos métodos o ambos para definir y administrar la IU de tu aplicación. Por ejemplo, podrías definir los diseños predeterminados de la aplicación en XML, incluidos los elementos de pantalla que aparecerán en ellos y sus propiedades. Luego podrías agregar código en tu aplicación para modificar el estado de los objetos de la pantalla, incluidos los declarados en XML, en tiempo de ejecución.

La ventaja de definir tu IU en XML es que te permite separar mejor la presentación de tu aplicación del código que controla su comportamiento. Tus descripciones de la IU son externas al código de tu aplicación, lo que significa que puedes modificarlo o adaptarlo sin tener que modificar el código fuente y volver a compilar. Por ejemplo, puedes crear diseños XML para diferente orientación de pantalla, diferentes tamaños de pantalla de dispositivos y diferentes idiomas. Además, definir el diseño en XML facilita la visualización de la estructura de tu IU, de modo que sea más fácil depurar problemas.

En general, el vocabulario XML para definir elementos de la IU sigue de cerca la estructura y la denominación de las clases y los métodos, en los que los nombres de los elementos coinciden con los nombres de las clases y los nombres de los atributos coinciden con los métodos. De hecho, la correspondencia generalmente es tan directa que puedes deducir qué atributo XML corresponde a un método de clase, o deducir qué clase corresponde a un elemento XML determinado. No obstante, ten en cuenta que no todo el vocabulario es idéntico. En algunos casos, hay pequeñas diferencias de denominación. Por ejemplo, el elemento **EditText** tiene un atributo **text** que puede asignarse usando el método **setText()** .

En los siguientes apartados trataremos aspectos importantes del diseño.

2.5.- Escribe en XML.

Al usar vocabulario XML de Android, puedes crear rápidamente diseños de IU y de los elementos de pantalla que contienen, de la misma manera que creas páginas web en HTML, con una serie de elementos anidados.

Cada fichero de diseño debe contener exactamente un elemento raíz, que debe ser un objeto **View** o **ViewGroup** . Una vez que hayas definido el elemento raíz, puedes agregar controles (***widgets***) u objetos de diseño adicionales como elementos secundarios para crear gradualmente una jerarquía de vistas que defina tu diseño.

Ejemplo

Aquí te mostramos un diseño XML que usa un elemento **LinearLayout** (subtipo de ViewGroup) vertical para incluir un elemento **TextView** (subtipo de View) y un elemento **Button** (subtipo de TextView):



Después de definir tu diseño en XML, guarda el fichero con la extensión **.xml** en el directorio **res/layout/** de tu proyecto de Android para que pueda compilarse correctamente.

2.6.- Carga el recurso XML.

Cuando compilas tu aplicación, cada fichero de diseño XML se compila en un recurso View. Debes cargar el recurso de diseño desde el código de tu aplicación, en tu implementación (redefinición) del método **onCreate()** de la clase Activity. Para hacerlo, llama a **setContentView()**, pásale la referencia a tu recurso de diseño en forma de: **R.layout.*layout_file_name***.

Ejemplo

Si tu diseño XML se guarda como **main_layout.xml**, lo cargarías para tu actividad de esta manera:

```
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main_layout);
}
```

El framework de Android llama al método de retrollamada **onCreate()** en tu actividad cuando se lanza la actividad.

2.7.- Atributos.

Todos los objetos **View** y **ViewGroup** admiten su propia variedad de atributos XML. Algunos atributos son específicos para un objeto **View** (por ejemplo, **TextView** admite el atributo **textSize**), pero a esos atributos también los heredan otros objetos **View** que podrían extender esta clase. Algunos son comunes para todos los objetos **View** ya que se heredan desde la clase **View** raíz (como el atributo **id**). Otros atributos se consideran "parámetros de diseño" y son atributos que describen ciertos tipos de orientación de diseño del objeto **View**, tal como lo define el objeto principal **ViewGroup** de ese objeto.

2.8.- ID.

Cualquier objeto **View** puede tener un **id** con número entero asociado a él para identificar de forma exclusiva la vista en un árbol. Cuando se compila la aplicación, este **id** se considera un número entero, pero el **id** generalmente se asigna en el archivo XML del diseño como un string, en el atributo **id**. Este es un atributo XML común a todos los objetos View (definidos por la clase **View**) y lo usarás con mucha frecuencia. La sintaxis para un **id** dentro de una etiqueta XML es la siguiente:

```
android:id="@+id/my_button"
```

El símbolo arroba (**@**) al comienzo del string indica al analizador de XML que debe analizar y expandir el resto de la cadena de **id** e identificarla como un recurso de **id**. El símbolo más (**+**) significa que es un nuevo nombre de recurso que se debe crear y agregar a nuestros recursos (en el archivo **R.java**). El framework de Android ofrece otros recursos de **id**. Al hacer referencia a un **id** de recurso de Android, no necesitas el símbolo más, pero debes agregar el espacio de nombres de paquete **android** de la siguiente manera:

```
android:id="@android:id/empty"
```

Con el espacio de nombres de paquete **android** establecido, ahora hacemos referencia a un **id** de la clase de recursos **android.R**, en lugar de la clase de recursos local.

Para crear vistas y hacer referencia a ellas desde la aplicación, puedes seguir este patrón común:

1. Definir una vista o un control (***widget***) en el archivo de diseño y asignarle un **id** único:

   Código

2. Luego, crear una instancia del objeto **View** y capturarlo desde el diseño (generalmente en el método onCreate()):

```
Button myButton = (Button) findViewById(R.id.my_button);
```

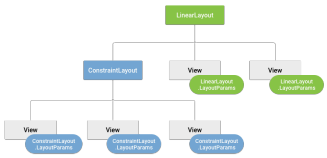
Definir **id** para objetos **View** es importante cuando se crea un **RelativeLayout** . En un diseño relativo, las vistas del mismo nivel pueden definir su diseño en función de otra vista del mismo nivel, que se identifica con un **id** único.

No es necesario que un **id** sea único en todo el árbol, pero debe ser único dentro de la parte del árbol en la que estás buscando (que a menudo puede ser el árbol completo, por lo que es mejor que, en lo posible, sea totalmente único).

2.9.- Parámetros de diseño.

Los atributos de diseño XML denominados **layout_*something*** definen parámetros de diseño para el objeto **View** que son adecuados para el objeto **ViewGroup** en el que reside.

Cada clase **ViewGroup** implementa una clase anidada que extiende ViewGroup.LayoutParams . Esta subclase contiene tipos de propiedad que definen el tamaño y la posición de cada vista secundaria, según resulte apropiado para el grupo de vistas. Como puedes ver en la figura, el grupo de vistas principal define parámetros de diseño para cada vista secundaria (incluido el grupo de vistas secundario).



Google Developers (Uso educativo nc)

Todos los grupos de vistas incluyen un ancho y una altura (**layout_width** y **layout_height**), y cada vista debe definirlos. Muchos **LayoutParams** también incluyen márgenes y bordes opcionales.

Puedes especificar el ancho y la altura con medidas exactas, aunque probablemente no quieras hacerlo con mucha frecuencia. Generalmente usarás una de estas constantes para establecer el ancho o la altura:

- ***wrap_content*** indica a tu vista que modifique su tamaño conforme a los requisitos de este contenido.
- ***match_parent*** indica a tu vista que se agrande tanto como lo permita su grupo de vistas principal.

En general, no se recomienda especificar el ancho y la altura de un diseño con unidades absolutas como píxeles. En cambio, el uso de medidas relativas como **unidades de píxeles independientes de la densidad** (***dp***), ***wrap_content***, o ***match_parent***, es un mejor enfoque, ya que ayuda a garantizar que tu aplicación se muestre correctamente en dispositivos con pantallas de diferentes tamaños.

2.10.- Posición del diseño.

La geometría de una vista es la de un rectángulo. Una vista tiene una ubicación, expresada como un par de coordenadas *izquierda* y *superior*, y dos dimensiones, expresadas como un ancho y una altura. La unidad para la ubicación y las dimensiones es el pixel.

Es posible recuperar la ubicación de una vista al invocar los métodos **getLeft()** y **getTop()** . El primero devuelve la coordenada izquierda, o X, del rectángulo que representa la vista. El segundo devuelve la coordenada superior, o Y, del rectángulo que representa la vista. Ambos métodos devuelven la ubicación de la vista respecto de su elemento primario. Por ejemplo, cuando **getLeft()** devuelve 20, significa que la vista se encuentra a 20 píxeles a la derecha del borde izquierdo de su elemento primario directo.

Además, se ofrecen varios métodos convenientes para evitar cálculos innecesarios, y se denominan **getRight()** y **getBottom()** . Estos métodos devuelven las coordenadas de los bordes derecho y superior del rectángulo que representa la vista. Por ejemplo, llamar a **getRight()** es similar al siguiente cálculo: **getLeft() + getWidth()**.

2.11.- Tamaño, relleno y márgenes.

El tamaño de una vista se expresa con un ancho y una altura. En realidad, una vista tiene dos pares de valores de ancho y altura.

El primer par se conoce como *ancho medido* y *altura medida*. Estas dimensiones definen cuán grande quiere ser una vista dentro de su elemento primario. Las dimensiones medidas se pueden obtener llamando a **getMeasuredWidth()** y a **getMeasuredHeight()** .

El segundo par se conoce simplemente como *ancho* y *altura*, o algunas veces *ancho de dibujo* y *altura de dibujo*. Estas dimensiones definen el tamaño real de la vista en la pantalla, al momento de dibujarlas y después del diseño. Estos valores pueden ser diferentes del ancho y la altura medidos, pero no necesariamente. El ancho y la altura se pueden obtener llamando a **getWidth()** y **getHeight()** .

Para medir estas dimensiones, una vista considera su relleno. El relleno se expresa en píxeles para las partes izquierda, superior, derecha e inferior de la vista. El relleno se puede usar para desplazar el contenido de la vista una determinada cantidad de píxeles. Por ejemplo, un relleno izquierdo de 2 empuja el contenido de la vista 2 píxeles hacia la derecha del borde izquierdo. El relleno se puede ajustar usando el método **setPadding(int, int, int, int)** y se puede consultar llamando a **getPaddingLeft()** , **getPaddingTop()** , **getPaddingRight()** y **getPaddingBottom()** .

Si bien una vista puede definir un relleno, no proporciona ningún tipo de soporte para márgenes. No obstante, los grupos de vistas sí lo proporcionan. Consulta **ViewGroup** y **ViewGroup.MarginLayoutParams** para obtener más información.

2.12.- Diseños comunes.

Cada subclase de la clase **ViewGroup** proporciona una manera única de mostrar las vistas que anidas en ella. Aquí te mostramos algunos de los tipos de diseño más comunes integrados en la plataforma Android y que podemos ver en la documentación de Android Developer.

Nota: Si bien puedes anidar uno o más diseños dentro de otro diseño para crear la presentación de tu IU, debes esforzarte por mantener tu jerarquía de diseño lo más sencilla posible. Tu diseño dibuja más rápido si tiene menos diseños anidados (una jerarquía de vistas ancha es mejor que una jerarquía de vista profunda).



Google Developers (Uso educativo nc)

1.- Diseño lineal

Un diseño que organiza sus elementos secundarios en una sola fila horizontal o vertical. Si la longitud de la ventana supera la longitud de la pantalla, crea una barra de desplazamiento.

2.- Diseño relativo



Google Developers (Uso educativo nc)

Te permite especificar la ubicación de los objetos secundarios en función de ellos mismos (el objeto secundario A a la izquierda del objeto secundario B) o en función del elemento primario (alineado con la parte superior del elemento primario).



Google Developers (Uso educativo nc)

3.- Vista web

Muestra páginas web.

3.- Controles de entrada y sus eventos.

Caso práctico

Además de conocer los distintos parámetros de diseño y cómo definirlos en una app de Android Studio mediante el lenguaje XML y los ficheros de recursos (carpeta res), es imprescindible conocer los mecanismos de la plataforma para interactuar con los usuarios.



Mikhail (pexels)

- ¿Alguno sabe decirme cómo se llama lo que provoca un usuario que interactúa con un control? -pregunta Ada.

- Un evento -responde María-. Le he leído en la documentación que nos has entregado.

- Muy bien María -le felicita Ada-. Realmente, cuando un usuario interactúa con un control provoca que se lancen eventos. Es necesario, en el diseño de la aplicación, aprender a capturar los eventos y programar las funciones que van a responder a los mismos.

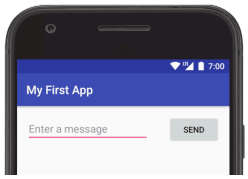
- Otra pregunta -vuelve a preguntar Ada-, ¿cómo se llaman las funciones que responden a los eventos?

- Controladores de eventos -responde Pedro.

- Veo que habéis estudiado la documentación -sonríe Ada-. Vamos a aprender cómo capturar los eventos y programar sus controladores.

3.1.- Controles de entrada.

Consultando la página de Android Developer podemos descubrir que los controles de entrada son los componentes interactivos de la interfaz de usuario de tu app. Android ofrece una gran variedad de controles que puedes usar en tu IU, como botones, campos de texto, barras de búsqueda, casillas de verificación, botones de zoom, botones de activación o desactivación y muchos más.



Google Developers (Uso educativo nc)

Agregar un control de entrada a tu IU es tan simple como agregar un elemento XML a tu **diseño XML** .

Ejemplo

Aquí te mostramos un diseño con un campo de texto y un botón:



Cada control de entrada admite un conjunto específico de eventos de entrada de modo que puedas gestionar eventos, como cuando el usuario introduce texto o pulsa un botón.

3.1.1.- Controles comunes.

Aquí te ofrecemos una lista con algunos de los controles comunes que puedes usar en tu app. Sigue los vínculos para obtener más información acerca de cómo usar cada uno.

Tipo de control	Clases relacionadas	Descripción
Botón	Button	Botón que el usuario puede presionar o en el que puede hacer clic para realizar una acción
Campo de texto	EditText AutoCompleteTextView	Campo de texto editable. Puedes usar el control AutoCompleteTextView para permitir al usuario introducir texto con sugerencias de autocompletado
Casilla de verificación	CheckBox	Conmutador de selección y deselección que el usuario puede activar o desactivar. Debes usar casillas de verificación (*checkboxes*) cuando presentes a los usuarios un grupo de opciones seleccionables que no sean mutuamente exclusivas.
Botón de selección	RadioGroup RadioButton	Similar a las casillas de verificación, excepto porque solo se puede seleccionar una opción en el grupo.
Botón para activar o desactivar	ToggleButton	Botón de activación y desactivación con un indicador luminoso.
Control de número	Spinner	Una lista desplegable que permite a los usuarios seleccionar un valor numérico de un conjunto.
Selectores	DatePicker TimePicker	Un cuadro de diálogo para que los usuarios seleccionen un valor individual para un conjunto con los botones de arriba y abajo o mediante un gesto de deslizamiento. Usa un control DatePicker para leer valores de fecha (mes, día, año) o un control TimePicker para leer valores de hora (hora, minuto) en modo 24 horas o AM/PM, a los que se les dará formato automáticamente de acuerdo con la configuración regional del usuario.

3.2.- Eventos de entrada.

Según Android Developer:

En Android, existe más de una forma de interceptar los eventos provocados por una interacción del usuario con tu aplicación. Al considerar los eventos dentro de tu interfaz de usuario, el enfoque consiste en capturar los eventos desde el objeto de vista específico con el que el usuario interactúa. La clase **View** proporciona los medios para hacerlo.

Dentro de las diversas clases de objetos **View** que usarás para componer tu diseño, quizá observes varios métodos de retrollamada (*callback*) públicos que pueden ser útiles para eventos de IU. La infraestructura (*framework*) de Android llama a estos métodos cuando la acción respectiva ocurre en el objeto vista. Por ejemplo, cuando el usuario interacciona con un objeto View (por ejemplo, un botón), se produce una llamada al método **onTouchEvent()** en ese objeto. Sin embargo, para interceptar esto, debes extender la clase y sobreescribir el método. No obstante, extender todas las subclases vista para manejar tal evento no sería práctico. Es por ello que la clase de vista también contiene una colección de interfaces anidadas con retrollamadas (*callbacks*) que puedes definir más fácilmente. Estas interfaces, llamadas **receptores de eventos** (*event listeners*), te permiten capturar la interacción del usuario con tu IU.

Si bien generalmente usarás los receptores de eventos para escuchar la interacción del usuario, es posible que en algún momento desees ampliar una clase View para crear un componente personalizado. Quizá desees ampliar la clase **Button** para crear algo más elaborado. En este caso, podrás definir los comportamientos de eventos predeterminados para tu clase utilizando los **controladores de eventos** (*event handlers*) de la clase.